プログラミング３ 講義資料

第10回 チーム開発②：基本設計

- 画面設計
- DB 設計
- システム構成

目次

- 画面・DB 設計
- システム構成と Issue 化

画面・DB 設計

今回の目的と到達目標

今回の目的

- 画面・DB を設計できる
- 構成図を書ける
- 設計から Issue を切る

到達目標

- [3] チーム開発での協調作業

基本設計の3点セット

設計項目	何を決める	主な成果物
画面設計	ユーザーに見える部分	画面遷移・モックアップ
DB 設計	データの持ち方	ER 図・テーブル定義
システム構成	サービス全体の構造	構成図・外部I/F一覧

画面設計の観点

- 必要な画面をリストアップ（ログイン / 一覧 / 詳細 / 編集 ...）
- 画面遷移を示す:

ログイン → 一覧 → 詳細 ⇄ 編集

- 各画面で: 表示するデータ / ユーザーが行う操作 / エラー時の挙動
- モックアップは紙・Figma・手書きどれでもよい
- 画面名と URL パスを合わせておくと設計・実装が揃う

DB 設計の基本

- データを **エンティティ（主要な名詞）** として洗い出す
- 例: ユーザー / 投稿 / コメント / タグ
- エンティティの属性を列に展開、関連を線で示す（ER 図）

テーブル	主なカラム
users	id / name / email
posts	id / user_id / title / body
comments	id / post_id / user_id / body

> 1対多・多対多の関係を明確にする

システム構成の考え方

- **クライアント**（ブラウザ / アプリ）— 誰がどこで使うか
- **サーバ**（Web / API）— 何を返すか
- **DB**（SQLite / PostgreSQL 等）— どこに保存するか
- **外部サービス**（認証 / 外部API / ストレージ）

クライアント ⇄ API サーバ ⇄ DB

- 通信プロトコル（HTTP / HTTPS）と形式（JSON）を明示

システム構成と Issue 化

設計から Issue への落とし込み

1. 画面ごとに Issue 作成（例: 「投稿一覧画面を実装」）
2. テーブルごとに Issue 作成（例: 「users テーブルを作成」）
3. 外部サービス連携も Issue に（例: 「認証API連携」）
4. 画面・テーブル・APIの対応関係を本文に書く
5. マイルストーン = 第11/12/13週 に割り振る

> 設計漏れは Issue に「TODO」ラベルで残しておくと忘れない

今週やること

Tips

- 基本設計書（画面 / DB / 構成）の3点をリポジトリに追加
- 設計レビューをチーム内で実施（軽く）
- 基本設計から Issue を新規作成 or 既存 Issue を更新
- 第2回 KPT 提出
- リベース前のブランチ運用ルールをチームで合意

今回のまとめ

- 基本設計の3点セット: 画面 / DB / システム構成
- エンティティ抽出 → ER 図、画面遷移 → URL 設計
- 設計から Issue を切って次週以降の実装につなぐ

今回カバーした MCC 項目

- V-D-4 コンピュータシステム

次回予告

- 第11回: チーム開発③ 詳細設計・Issue/PR 本格運用